peer-to-peer idea

protocols we've talked about so far:

small set of machines in charge

- single set of root DNS server
- single set of HTTP reverse proxies browser contacts
- ...

maybe more democratic idea is that everyone can run part of service

no central authority that controls system

BitTorrent

tool for lots of people downloading big file

- example: install disk for Linux distribution
- (yes, there are some not-so-great examples)

goal: everyone helps each other download the file

- rather than making it the job of a few overloaded servers
- take advantage of 'spare' bandwidth

can also use these ideas for transferring files within a network

‘torrent’ files

torrent files:

- a summary of the big file
- ‘tracker’ information

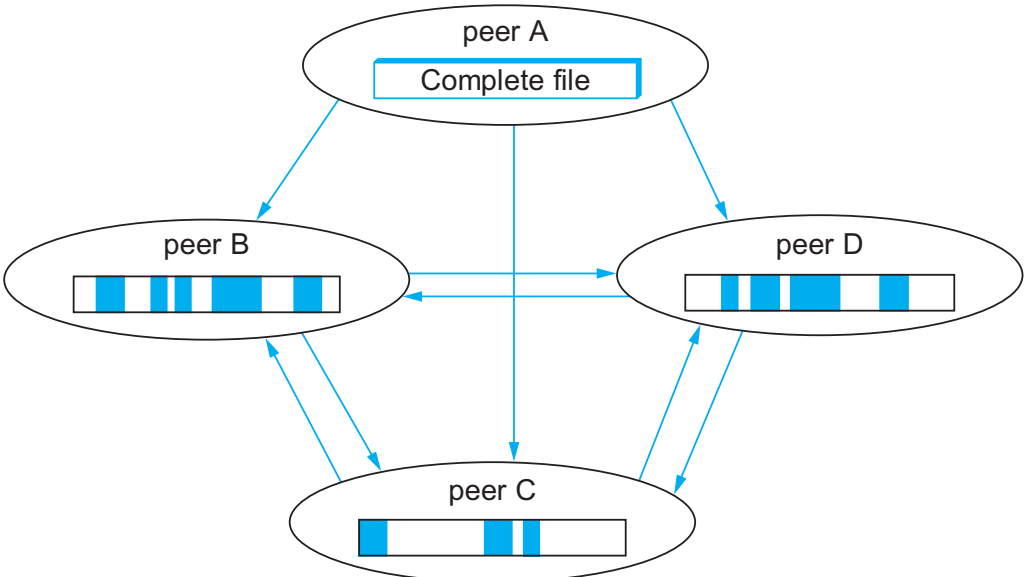
list of the cryptographic hashes of every X bytes of the file

- typical X: 256KB-1MB

then: can verify if we have piece Y

...so don’t care where it comes from

download from everyone (1)



download from everyone (2)

if A has whole file (parts 1–12), B has parts 1, 3, 5; C has parts 2, 4, 7.

then possible plan:

get parts 5, 6, 8, 9, 10, 11, 12 from A
get parts 1, 3, 5 from B
get parts 2, 4, 7 from C

but better than that:

B and C are also downloading file, probably they'll have more parts later!

connecting to peers

nodes connect to a set of peers

- later: getting a list

exchange info about what pieces are available

a planning question

if A has whole file (parts 1–12), B has parts 1, 3, 5; C has parts 2, 4, 7.

could spend a lot of effort planning who sends what to whom...

- take advantage of differences in speeds between nodes

but...

- no central authority
- nodes drop in/out of network
- lots of nodes in practice

so simpler typical solution: get random available piece

randomness allows sharing

suppose A and B are both downloading a file

if they both did it in same order — only sharing in one direction

- also true if they both prioritize in the same way

if they do it in random order

- if both have 20% of file, probably only around 20% overlap by chance

freeriding

user goal: one wants to download file

...often little incentive to help others download

could just reject requests to download...

or make them go superslow

tit-for-tat

algorithm to encourage uploading:

when choosing who to upload to...

prefer requests for nodes that have uploaded to you

'choke' nodes that have not in a while

- at least if you have 'better' choices to upload to

flow control

even ignoring tit-for-tat...

don't want to just download from/upload to 10k peers at once

- lots of overhead (TCP, otherwise) relative to data siez

signaling protocol:

- uploaders indicate peer choked (can get data)/unchoked (cannot)
- downloaders indicate they are interested/not interested
- only transfer if interested + unchoked

trackers

still need to find peers

original BitTorrent solution: trackers run over HTTP

peers periodically make HTTP request with their info

tracker maintains database of recent peers

returns a list of possible peers and their addresses

later: decentralized solution

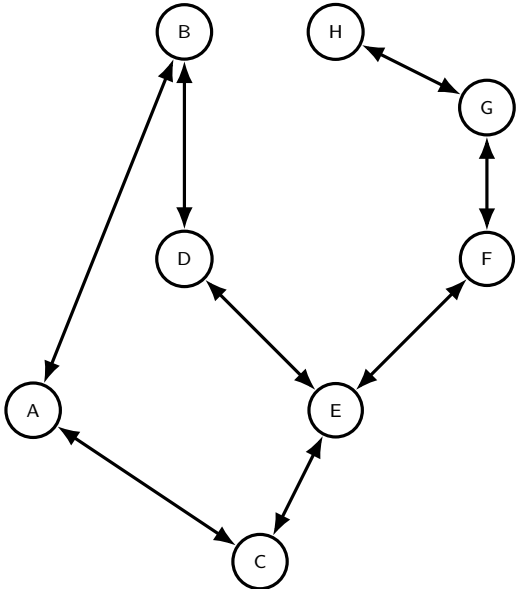
distributed search

common peer-to-peer to problem

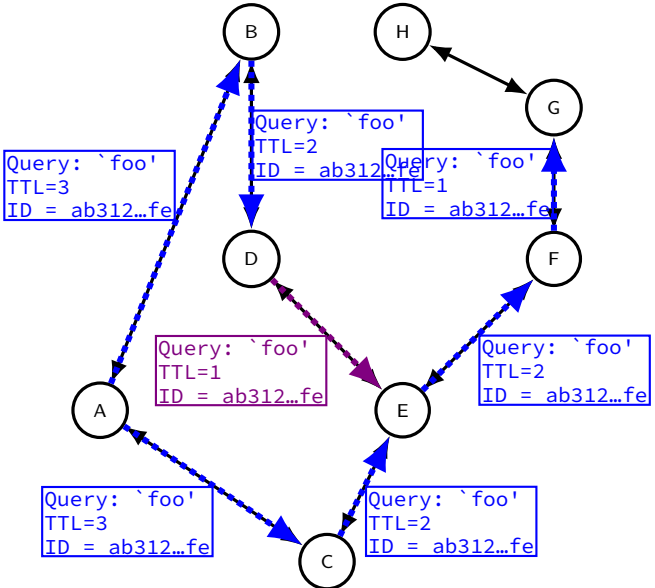
how do you find out who has some data?

- database entry
- list of peers for file
- files whose name contains string

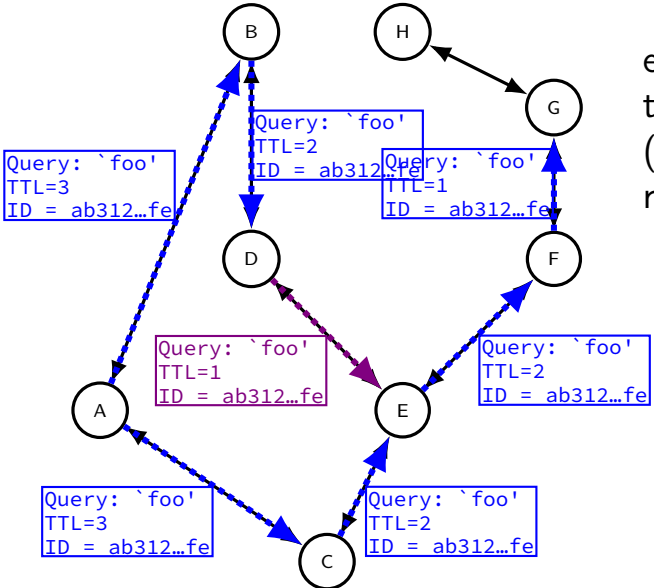
flood-to-search



flood-to-search

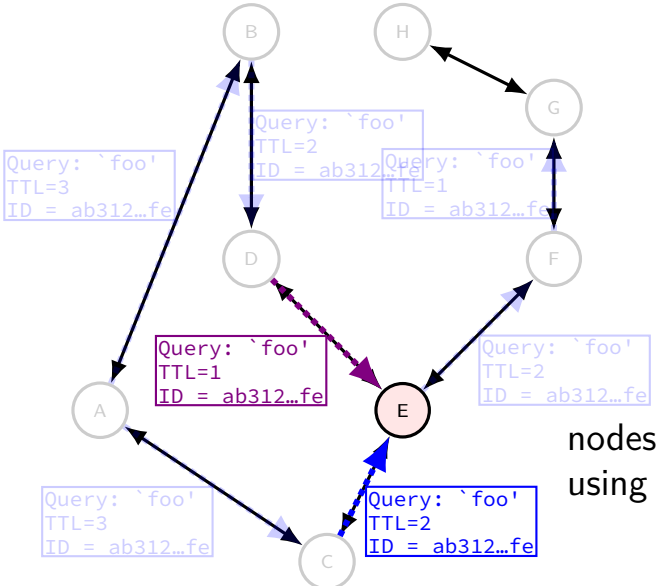


flood-to-search



every node forwards to neighbors (similar to link-state routing flooding)

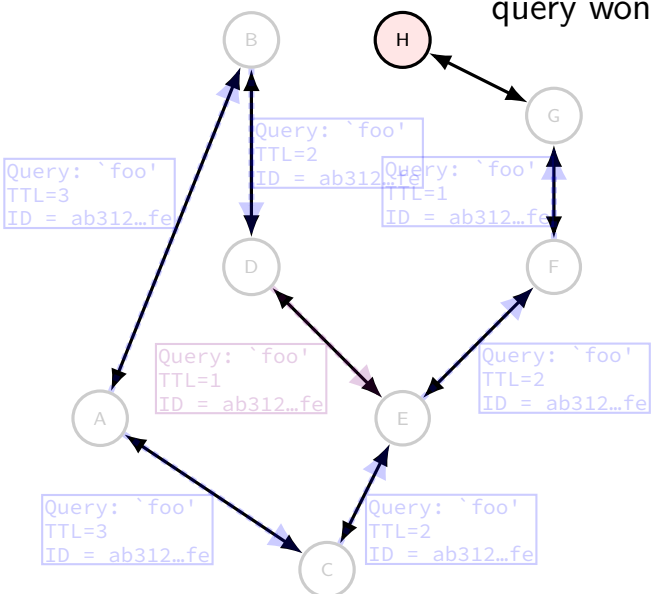
flood-to-search



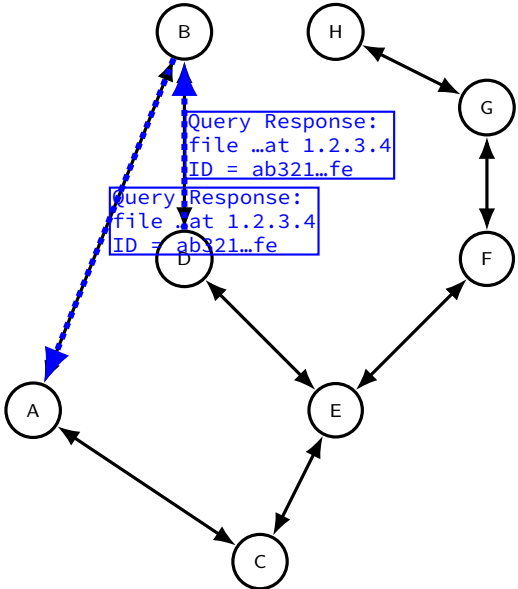
nodes reject duplicates using query ID

flood-to-search

if TTL not high enough
query won't get to all nodes



flood-to-search



nodes with response(s) send back

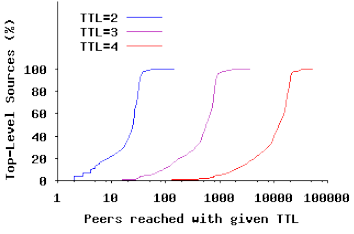
small worlds

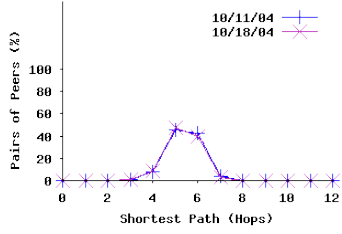
might seem we need high TTL, so queries slow...

but not really:

- typical nodes have 10s of peers, not 1 or 2
- rouhgly random connections make it likely paths are short

some figures from paper by Stutzbach, Rajaie, and Sen in 2005





not scaling

flood-to-search pretty inefficient with large network

Gnutella did some optimizations that undecentralize network some:

split nodes into ultrapeers and leafs
ultrapeers copy whole index for nearby leafs

distributed hash tables

to avoid flood to search when searching:

for every search term X, choose node to hold results for it

how to choose node?

- hash(X) closest to hash(node ID)

what if node goes down?

- next-closest hash(node ID)s act as backup

how to find specific node?

- small network: full index of nodes
- otherwise: nodes track nearby hash(node ID) ...
- then forward to node with closer hash(node ID)

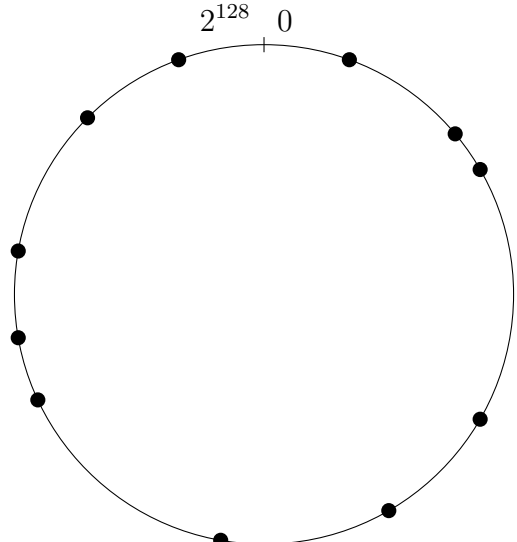
next slides: one DHT design

a bunch of DHT designs out there

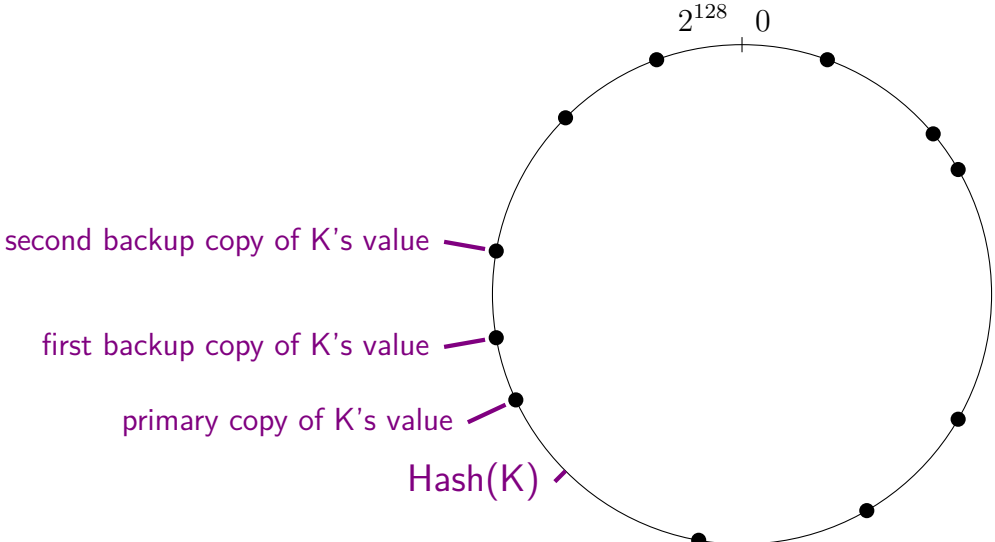
differ primarily in of how keys are assigned to nodes

will show one example design

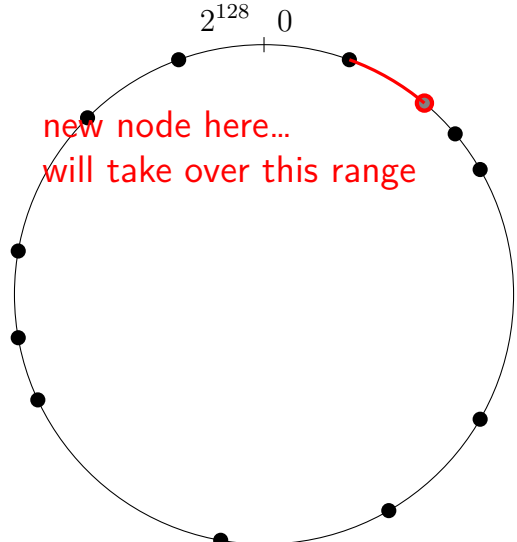
one possible DHT design



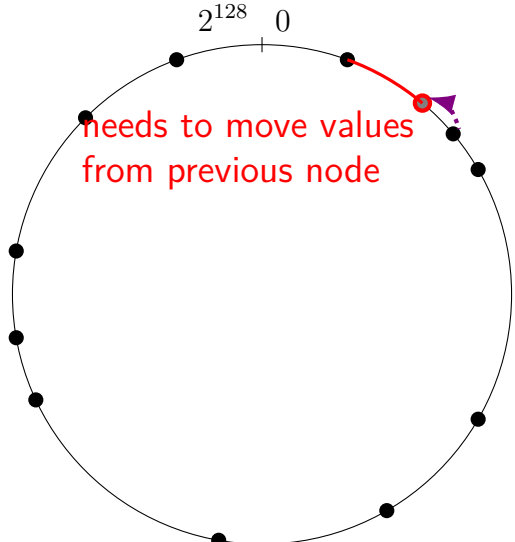
one possible DHT design



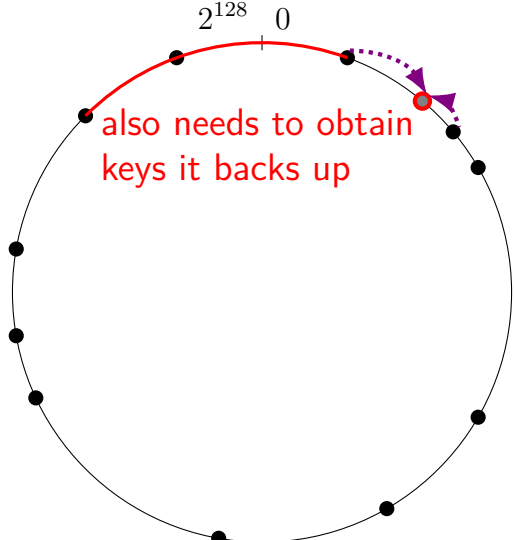
one possible DHT design



one possible DHT design

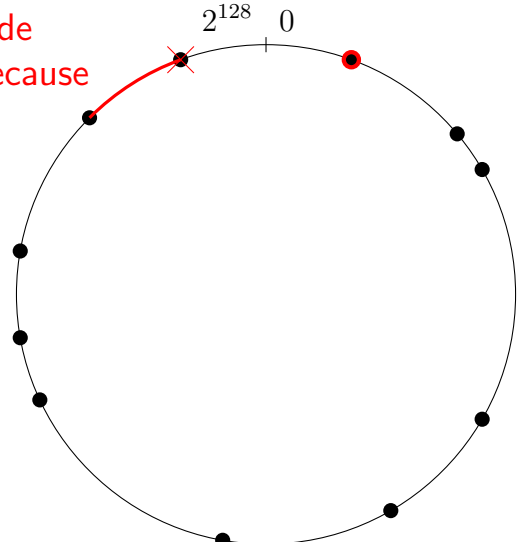


one possible DHT design



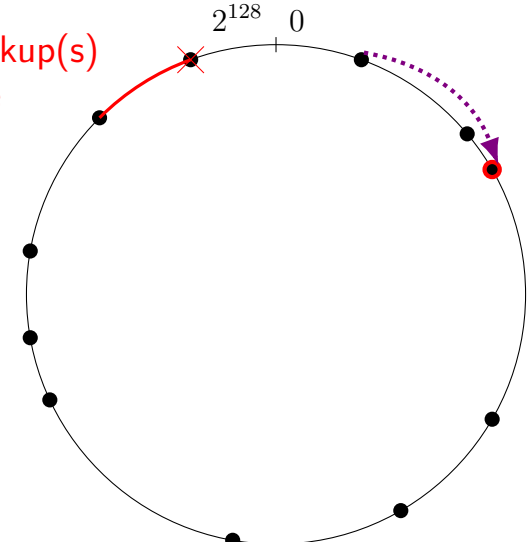
one possible DHT design

if this node goes down...
its keys assigned to next node
which already has a copy because
it is the first backup



one possible DHT design

if this node goes down...
also need to send new last backup(s)
to get copy of its former range



DHT node tracking

every node tracks its 'nearby' (by Hash(node ID)) neighbors

- at least immediate successor...
- but probably more to handle failures

node needs to react if neighbor goes down by taking over for its keys

finding objects

most simple approach:

- everyone sent full list of nodes and their IP/port/etc.
- works for small DHT-like design
- example: DynamoDB (in-datacenter database)

simple distributed approach:

- everyone has successor node
- go around the ring until you find the right node

shortcuts

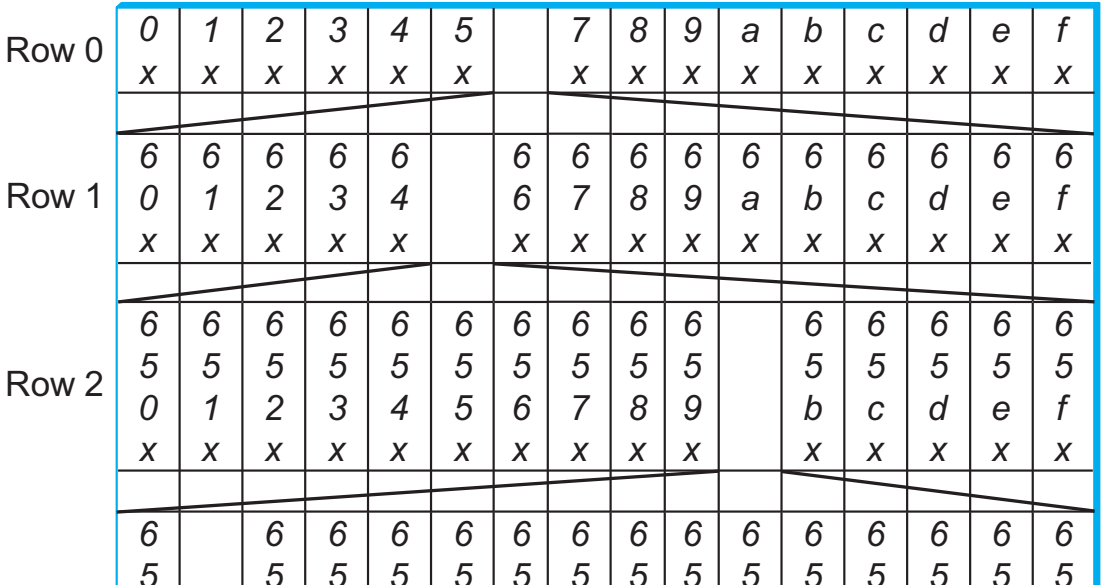
problem with simple approach: N steps

optimization:

> if we know a few other nodes...
> can start at one that's closer

question: which other nodes to know

‘routing tables’ / ‘finger tables’



textbook routing table idea

if my ID is 0x65a1fc...

track something like

node for 0x0???..., 0x1???..., 0x2???..., ...
node for 0x60??..., 0x61??..., 0x62??..., ...
node for 0x650?..., 0x6510..., 0x652?..., ...
node for 0x6510..., 0x6511..., 0x6512..., ...
...

(not the only possible choice for this)

DHT variations

different notion of ‘closest’

- next hash value with wraparound not only option

different ideas of how to routing/finger tables

backup slides